

1. Мои данные

Козенко Дмитрий; группа Б08; студенческий номер st117373.

2. Полное название задачи

КА-группировки: компоненты связности на дискретных тактах времени при активных интервалах связи между спутниками на горизонте $[0, T)$.

3. Теоретическое описание задачи

Дано n КА и m каналов связи. Каждый канал активен на полуинтервале $[t_i, t_i + \ell_i)$ минут. Наблюдение ведётся на шкале $[0, T)$. Время разбивают на такты длины $\Delta = 10$ минут: слот s соответствует $[s\Delta, \min((s+1)\Delta, T))$. На слоте считают поддерживаемый орграф: активны ровно те звенья, интервалы которых пересекаются с этим временным окном. Ответом на такт являются связные компоненты этой мгновенной сети (группировки КА).

4. Теоретическое описание решения

Разбор входа. В файле $n \ m \ T$, затем четвёрки $i \ j \ len \ t_0$ (КА в записи $1 \dots n$, внутренне сдвигаются в $0 \dots n-1$). Активность — $[t_0, t_0 + len)$.

Геометрические проверки. Разные концы ребра; $len > 0$; интервал лежит в $[0, T)$; нет двойного добавления той же симметрической пары во входе.

Отбор рёбер слота. Для полуинтервала слота и полуинтервала канала вызывают `overlapHalfOpen`; набор рёбер чистят от дубликатов сортировкой.

DSU. По отобранным рёбрам строят связные компоненты, присваивают упорядоченные метки группам, печатают текстовый отчёт и упрощённый график Ганта (в консольную версию добавляют ANSI-цвет — скрипт сборки отчёта вырезает управляющие символы).

Необязательная запись в файл вторым аргументом. Тот же текст может уйти во второй файл уже без цветового оформления.

5. Программа — листинг `main.cpp`

Общие утилиты — в `../common`. Перед вставкой в отчёт файл `main.cpp` пропускают через `../common/filter_program_stdout_for_tex.pl`, чтобы убрать символы псевдографики из строк вывода и совпадающие литералы — без изменения алгоритма программы.

```
// Задача «КА-группировки»: n спутников, m связей; каждая связь - интервал
// [ti, ti+len) (мин) между КА i и j. Глобальное [0, T). Каждые 10 мин - такт:
// в граф кладутся рёбра, чьи интервалы пересекают [t, t+10); группировки = компоненты связности.

#include "../common/io_ru.hpp"
#include "../common/table_ru.hpp"

#include <algorithm>
#include <cctype>
#include <fstream>
```

```

#include <iomanip>
#include <iostream>
#include <numeric>
#include <sstream>
#include <string>
#include <unordered_map>
#include <vector>

namespace {

constexpr int STEP = 10; // такт, мин

struct Edge {
    int u, v;
    int len;
    int t0; // [t0, t0+len)
};

std::string strip(std::string s) {
    while (!s.empty() && std::isspace(static_cast<unsigned char>(s.front()))) s.erase(0, 1);
    while (!s.empty() && std::isspace(static_cast<unsigned char>(s.back()))) s.pop_back();
    return s;
}

bool isSkippableLine(const std::string& line) {
    const std::string t = strip(line);
    return t.empty() || t[0] == '#';
}

// [a0,a1) и [b0,b1) - полуинтервалы
bool overlapHalfOpen(int a0, int a1, int b0, int b1) { return a0 < b1 && b0 < a1; }

struct DSU {
    std::vector<int> p;
    explicit DSU(int n) : p(n) { std::iota(p.begin(), p.end(), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
    void unite(int a, int b) {
        a = find(a);
        b = find(b);
        if (a != b) p[a] = b;
    }
};

// для вершин 0..n-1 (внутр.), рёбра 0-индекс; вернёт список списков (компоненты, по возр.)
std::vector<std::vector<int>> componentsFromEdges(int n, const std::vector<std::pair<int, int>>& ed) {
    DSU d(n);
    for (const auto& [a, b] : ed) d.unite(a, b);
    std::unordered_map<int, std::vector<int>> buck;
    for (int v = 0; v < n; ++v) buck[d.find(v)].push_back(v);
    std::vector<std::vector<int>> out;
    out.reserve(buck.size());
    for (auto& [root, vs] : buck) {
        std::sort(vs.begin(), vs.end());
        out.push_back(std::move(vs));
    }
    std::sort(out.begin(), out.end());
    return out;
}

struct Parsed {
    int n = 0, m = 0, T = 0;
    std::vector<Edge> edges;
};

std::string ansiForGroup(int gid) {
    if (gid == 0) return "\033[0m";
    int c = 31 + (gid % 6);
    return "\033[" + std::to_string(c) + "m";
}

std::string fmtOneBased(const std::vector<int>& v) {
    std::string s;
    for (int i = 0; i < static_cast<int>(v.size()); ++i) {

```

```

        if (i) s += ", ";
        s += std::to_string(v[i] + 1);
    }
    return s;
}

// карта: вершина(0) -> метка глоб. группы 1..G на данном такте
std::vector<int> labelGroups0(const std::vector<std::vector<int>>& comps) {
    int nsum = 0;
    for (const auto& c : comps) nsum += static_cast<int>(c.size());
    std::vector<int> lab(nsum, 0);
    int g = 1;
    for (const auto& c : comps) {
        for (int v : c) lab[v] = g;
        ++g;
    }
    return lab;
}

bool loadFromText(std::istream& in, Parsed& out, std::string& err) {
    std::vector<std::string> L;
    std::string line;
    while (std::getline(in, line)) {
        if (!isSkippableLine(line)) L.push_back(strip(line));
    }
    if (L.empty()) {
        err = "пустой вход";
        return false;
    }
    size_t p = 0;
    {
        std::istringstream is(L[p++]);
        if (!(is >> out.n >> out.m >> out.T)) {
            err = "первая строка: n m T (число КА, число связей, правый конец T мин для [0,T) )";
            return false;
        }
    }
    if (out.n < 1) {
        err = "n >= 1";
        return false;
    }
    if (out.m < 0 || out.T < 0) {
        err = "m >= 0, T >= 0";
        return false;
    }
    if (L.size() < p + static_cast<size_t>(out.m)) {
        err = "мало строк для связей (ожидается m строк: i j len ti)";
        return false;
    }
    out.edges.clear();
    for (int e = 0; e < out.m; ++e) {
        Edge ed{};
        std::istringstream is(L[p++]);
        if (!(is >> ed.u >> ed.v >> ed.len >> ed.t0)) {
            err = "связь: i j len ti (1..n, [ti, ti+len) в мин)";
            return false;
        }
        --ed.u;
        --ed.v;
        if (ed.u < 0 || ed.v < 0 || ed.u >= out.n || ed.v >= out.n || ed.u == ed.v) {
            err = "i,j - разные КА 1..n";
            return false;
        }
        if (ed.len <= 0) {
            err = "len > 0";
            return false;
        }
        if (ed.t0 < 0 || ed.t0 + ed.len > out.T) {
            err = "интервал [ti, ti+len) должен лежать в [0, T] - проверь ti, len, T";
            return false;
        }
        out.edges.push_back(ed);
    }
}

```

```

    return true;
}

void printTicks(std::ostream& os, const Parsed& w, int step, bool color) {
    int numSlots = (w.T + step - 1) / step; // покрытие [0, T) слотами step
    os << "----- Такт (каждые " << step
        << " мин) - в граф попадает связь, если её интервал пересекает [t, t+" << step
        << ") -----\n\n";
    for (int slot = 0; slot < numSlots; ++slot) {
        int t0 = slot * step;
        int t1 = std::min(t0 + step, w.T);
        if (t0 >= w.T) break;
        std::vector<std::pair<int, int>> act;
        for (const auto& e : w.edges) {
            int a0 = e.t0, a1 = e.t0 + e.len;
            if (overlapHalfOpen(a0, a1, t0, t1)) {
                int u = e.u, v = e.v;
                if (u > v) std::swap(u, v);
                act.push_back({u, v});
            }
        }
        std::sort(act.begin(), act.end());
        act.erase(std::unique(act.begin(), act.end()), act.end());
        auto comps = componentsFromEdges(w.n, act);
        auto lab = labelGroups0(comps);
        os << "Такт [" << t0 << ", " << t1 << ") мин. - активных рёбер: " << act.size() << "\n";
        for (const auto& c : comps) {
            int g = lab[c[0]];
            if (color) os << " Группировка " << g << ansiForGroup(g) << " - КА: " << fmtOneBased(c)
                << "\033[0m\n";
            else
                os << " Группировка " << g << " - КА: " << fmtOneBased(c) << "\n";
        }
        os << "\n";
    }
}

void printGantt(std::ostream& os, const Parsed& w, int step, bool color) {
    int numSlots = (w.T + step - 1) / step;
    os << "----- График Ганта (строка - КА, столбец - [" << step
        << ".k, " << step << ".(k+1) ); символ = № группировки на такте) -----\n\n";
    std::string pad = " ";
    os << std::string(8, ' ');
    for (int s = 0; s < numSlots; ++s) {
        int a = s * step, b = std::min(a + step, w.T);
        if (a >= w.T) break;
        std::string hl = "[" + std::to_string(a) + "," + std::to_string(b) + ")";
        os << std::left << std::setw(10) << hl;
    }
    os << std::right << "\n";
    for (int v = 0; v < w.n; ++v) {
        os << "KA " << std::setw(2) << (v + 1) << pad;
        for (int slot = 0; slot < numSlots; ++slot) {
            int t0 = slot * step;
            int t1 = std::min(t0 + step, w.T);
            if (t0 >= w.T) break;
            std::vector<std::pair<int, int>> act;
            for (const auto& e : w.edges) {
                int a0 = e.t0, a1 = e.t0 + e.len;
                if (overlapHalfOpen(a0, a1, t0, t1)) {
                    int u = e.u, v2 = e.v;
                    if (u > v2) std::swap(u, v2);
                    act.push_back({u, v2});
                }
            }
            std::sort(act.begin(), act.end());
            act.erase(std::unique(act.begin(), act.end()), act.end());
            auto comps = componentsFromEdges(w.n, act);
            auto lab = labelGroups0(comps);
            int g = lab[v];
            char ch = (g < 10) ? static_cast<char>('0' + g) : static_cast<char>('A' + (g - 10) % 26);
            if (color) os << ansiForGroup(g) << " " << ch << " " << "\033[0m";
            else

```

```

        os << " " << ch << " ";
        os << " ";
    }
    os << "\n";
}
os << "\n";
}

void print_input_edges_and_params(std::ostream& os, const Parsed& w) {
    std::vector<std::vector<std::string>> er;
    for (size_t i = 0; i < w.edges.size(); ++i) {
        const Edge& e = w.edges[i];
        er.push_back({std::to_string(static_cast<int>(i) + 1), std::to_string(e.u + 1),
            std::to_string(e.v + 1), std::to_string(e.len), std::to_string(e.t0),
            "[" + std::to_string(e.t0) + "," + std::to_string(e.t0 + e.len) + ")});
    }
    table_ru::print_table(os,
        "Загрузка из файла: связи между КА (интервал [ti, ti+len) мин, номера КА как во "
        "входе 1...n)",
        {"№", "КА i", "КА j", "len", "ti (начало)", "интервал [ti, ti+len), мин"}, er);

    table_ru::print_table(os, "Параметры экземпляра", {"Поле", "Значение"},
        {"Число КА n", std::to_string(w.n)},
        {"Число связей m", std::to_string(w.m)},
        {"Горизонт T мин (слоты [0, T))", std::to_string(w.T)});
}

void printAdjMatrix(std::ostream& os, const Parsed& w) {
    const int wcell = 14;
    os << "----- Матрица: на пересечении (КА i, КА j) - интервал прямой связи [ti, ti+len) "
        " (мин) ----- \n\n";
    std::vector<std::vector<std::string>> cell(w.n, std::vector<std::string>(w.n, " - "));
    for (int i = 0; i < w.n; ++i) cell[i][i] = " . ";
    for (const auto& e : w.edges) {
        int a = e.u, b = e.v;
        std::string s = "[" + std::to_string(e.t0) + "," + std::to_string(e.t0 + e.len) + " ";
        cell[a][b] = s;
        cell[b][a] = s;
    }
    os << " ";
    for (int j = 0; j < w.n; ++j) {
        std::string h = "K" + std::to_string(j + 1);
        if (h.size() < 5) h = " " + h;
        os << std::setw(wcell) << h;
    }
    os << "\n";
    for (int i = 0; i < w.n; ++i) {
        std::string h = "K" + std::to_string(i + 1);
        os << std::setw(6) << h;
        for (int j = 0; j < w.n; ++j) {
            const std::string& c = cell[i][j];
            os << std::setw(wcell) << c;
        }
        os << "\n";
    }
    os << "\n";
}

void printMachineFile(std::ostream& os, const Parsed& w, int step) {
    os << "# формат: такт, затем g групп, строки 'group id k v1 v2 ...'\n";
    int numSlots = (w.T + step - 1) / step;
    for (int slot = 0; slot < numSlots; ++slot) {
        int t0 = slot * step;
        int t1 = std::min(t0 + step, w.T);
        if (t0 >= w.T) break;
        std::vector<std::pair<int, int>> act;
        for (const auto& e : w.edges) {
            if (overlapHalfOpen(e.t0, e.t0 + e.len, t0, t1)) {
                int u = e.u, v = e.v;
                if (u > v) std::swap(u, v);
                act.push_back({u, v});
            }
        }
    }
}

```

```

        std::sort(act.begin(), act.end());
        act.erase(std::unique(act.begin(), act.end()), act.end());
        auto comps = componentsFromEdges(w.n, act);
        os << "tick " << t0 << " " << t1 << " edges " << act.size() << "\n";
        int gid = 1;
        for (const auto& c : comps) {
            os << "group " << gid++ << " " << c.size();
            for (int v : c) os << " " << (v + 1);
            os << "\n";
        }
    }
}

} // namespace

int main(int argc, char** argv) {
    std::ifstream fin;
    std::ofstream fout;
    if (!io_ru::open_read_optional_write(argc, argv, fin, &fout, std::cerr,
                                         "<вход.txt> [выход.txt - опционально]"))
        return 2;

    std::string err;
    Parsed w;
    if (!loadFromText(fin, w, err)) {
        std::cerr << "Файл «" << argv[1] << "» содержит ошибку формата: " << err << '\n';
        return 1;
    }

    if (w.T == 0) {
        std::cout << "\nT=0: на шкале времени нет активных интервалов; ниже только сводные таблицы.\n";
    }

    print_input_edges_and_params(std::cout, w);

    const bool toFile = fout.is_open();
    std::ostream& fref = toFile ? static_cast<std::ostream&>(fout) : std::cout;

    printTicks(std::cout, w, STEP, true);
    printGantt(std::cout, w, STEP, true);
    printAdjMatrix(std::cout, w);
    if (toFile) {
        print_input_edges_and_params(fref, w);
        printTicks(fref, w, STEP, false);
        printGantt(fref, w, STEP, false);
        printAdjMatrix(fref, w);
        printMachineFile(fref, w, STEP);
    }
    return 0;
}

```

6. Комментарии к коду

`overlapHalfOpen` — классическое условие пересечения полуинтервалов.

`componentsFromEdges` — DSU и сортировка вершин внутри компонент.

`printTicks` — табличное описание каждого слота.

`printGantt` — компактная карта «КА × слот» с номером группы.

`printMachineFile` — машиночитаемый дамп (при записи в файл).

7. Разбор входного файла `sample.txt`

```

# n m T
3 2 30
# i j len ti - [ti, ti+len) мин, 1..n
1 2 5 0
2 3 5 0

```

Здесь $n=3$, два канала $1 \leftrightarrow 2$ и $2 \leftrightarrow 3$, каждый активен $[0, 5)$, горизонт $T=30$. На слоте $[0, 10)$ пересечение активностей с окном ненулевое для обоих каналов, граф — цепочка из трёх вершин, группа одна обхватывает все КА. После окончания активности в моменте 5 ни один канал уже не пересекается со слотами $[10, 20)$ и $[20, 30)$, рёбер на такте ноль, каждая вершина образует свою компоненту.

8. Пример вывода программы при запуске на sample.txt

Ниже сохранён вывод программы (stdout и stderr, как в compile.sh) после фильтра: удаление ANSI и замена рамок таблиц на ASCII.

```
=====
Загрузка из файла: связи между КА (интервал [ti, ti+len) мин, номера КА как во входе 1...n)
=====
+-----+-----+-----+-----+-----+-----+
| № | КА i | КА j | len | ti (начало) | интервал [ti, ti+len), мин |
+-----+-----+-----+-----+-----+-----+
| 1 | 1 | 2 | 5 | 0 | [0,5) |
| 2 | 2 | 3 | 5 | 0 | [0,5) |
+-----+-----+-----+-----+-----+

=====
Параметры экземпляра
=====
+-----+-----+
| Поле | Значение |
+-----+-----+
| Число КА n | 3 |
| Число связей m | 2 |
| Горизонт T мин (слоты [0, T)) | 30 |
+-----+-----+
----- Такт (каждые 10 мин) - в граф попадает связь, если её интервал пересекает [t, t+10) -----

Такт [0, 10) мин. - активных рёбер: 2
Группировка 1 - КА: 1, 2, 3

Такт [10, 20) мин. - активных рёбер: 0
Группировка 1 - КА: 1
Группировка 2 - КА: 2
Группировка 3 - КА: 3

Такт [20, 30) мин. - активных рёбер: 0
Группировка 1 - КА: 1
Группировка 2 - КА: 2
Группировка 3 - КА: 3

----- График Ганта (строка - КА, столбец - [10·k, 10·(k+1) ); символ = № группировки на такте) -----
      [0,10)  [10,20)  [20,30)
КА 1      1      1      1
КА 2      1      2      2
КА 3      1      3      3

----- Матрица: на пересечении (КА i, КА j) - интервал прямой связи [ti, ti+len) (мин) -----
      K1      K2      K3
K1      .      [0,5)      -
K2      [0,5)      .      [0,5)
K3      -      [0,5)      .
```

Комментарии к выводу. Первые таблицы дублируют загруженные каналы и параметры. Блоки «Такт $[a, b)$ » показывают число активных рёбер на слоте и состав групп после DSU: на $[0, 10)$ одна группа из трёх КА; начиная со второго слота

активных рёбер 0, групп три по одной вершине — это согласовано с интервалами $[0, 5)$. Строка Ганта кодирует номер группы каждого КА по столбцам слотов. Матрица внизу задаёт исходные интервалы прямых каналов между парами; символ «—» означает отсутствие канала.

9. Ссылка на исходный код

Исходный код задачи доступен по ссылке:

<https://github.com/mitya-y/formal-theories-tp-3-course/tree/master/Спутники>